

Dragonfly Topology:

Dragonfly is a hierarchical, **low-diameter interconnection network** designed to minimize the number of hops required for communication between nodes while using relatively few high-cost global links. It was proposed by Kim et al. (ISCA 2008) for large-scale HPC systems and is widely studied for both supercomputers and Network-on-Chip (NoC) research.

Unlike a Mesh, where packets traverse many intermediate routers, Dragonfly organizes routers into **groups (clusters)** connected by high-bandwidth global links, allowing packets to reach distant destinations in only a few hops.

Primary objective: Reduce communication latency and network energy by minimizing average hop count.

Architecture:

Dragonfly has a **hierarchical organization** consisting of:

- **Processing Nodes (PEs)** connected to routers
- **Routers** connected locally within a group
- **Groups (Clusters)** connected through global links

Communication occurs at two levels:

- ❖ Intra-group communication
 - Routers inside the same group are connected.
 - Packets destined within the group remain local.
- ❖ Inter-group communication
 - If the destination is outside the current group, the packet:
 - Travels to a router with the required global link.
 - Crosses a single global connection.
 - Reaches the destination group.
 - Is routed locally to the destination node.

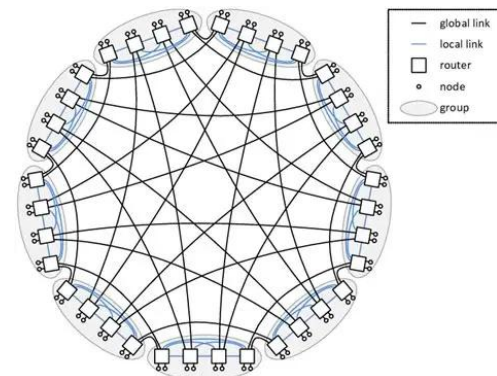
This hierarchy significantly reduces network diameter compared to Mesh or Torus.

Advantages

Feature	Why
Very low diameter	Few hops
Lower latency	Short routes
Lower NoC energy	Fewer routers crossed
High scalability	Thousands of nodes
High bandwidth	Many parallel global links

Limitations

Limitation	Reason
Complex wiring	Global links are long
Difficult routing	Need careful algorithms
Expensive layout	Many long interconnects
Harder to design	More parameters than Mesh
Booksim limitation	Only n=1 implemented here



Summary of Key Formulas:

Parameter	Formula
Router radix	$(4p-1)$
Routers per group	$(a=2p)$
Groups	$(g=2p^2+1)$
Total nodes	$(N=apg=2p^2(2p^2+1))$
Total routers	$(R=N/p)$
Nodes per group	(ap)
Router ID	NodeID / p
Group ID	$\text{RouterID} / a$
Channels	$(R(3p-1))$

Feature	General Dragonfly	Booksim Dragonfly
Dimensions	Multiple	Only 1
Groups	Arbitrary	Formula-based
Routing	Many adaptive algorithms	Fixed implementation
Global links	Flexible	Fixed computation
Router radix	User configurable	Derived automatically
Hierarchy	Multi-level	Single level
Complexity	Very high	Simplified

Conclusion:

Dragonfly is a hierarchical topology that reduces communication latency and energy by organizing routers into locally connected groups linked through a small number of global channels. Compared to Mesh, it achieves significantly lower average hop counts, making it attractive for large-scale NoCs and HPC systems. The trade-off is increased router complexity and more challenging physical implementation. The Booksim implementation models a simplified version of Dragonfly, making it suitable for evaluating routing algorithms and communication behavior while keeping the implementation manageable.

KNCube(Mesh/Torus)Topology:

The **KNCube topology** is a regular **n-dimensional direct network** in which routers are arranged as a multidimensional grid. Each router connects to its neighboring routers along each dimension and to one or more processing nodes.

In Booksim, the KNCube implementation is used to generate both:

- **Mesh topology** (mesh = true)
- **Torus topology** (mesh = false)

The only difference lies in how the routers at the boundaries are connected.

Mesh leaves edge routers open, whereas Torus wraps the edges around to form a closed network.

Architecture:

The network is characterized by two parameters:

- k = Number of routers along each dimension
- n = Number of dimensions

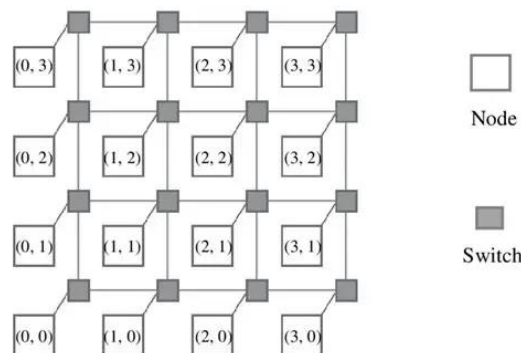
Booksim computes:

$$\text{Total Routers} = k^n$$

Since one processing node is attached to each router,

$$\text{Total Nodes} = k^n$$

❖ Mesh Architecture



Boundary routers terminate at the edges.

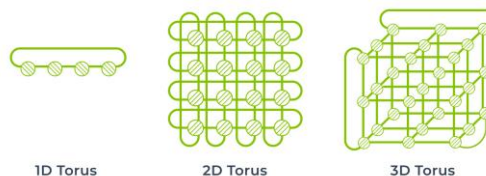
No wrap-around links exist.

$$\text{Network Diameter, } D = 2(k-1)$$

❖ Torus Architecture

Torus Topology

Direct-Connection Network Topology Type



Left connects to right.

Top connects to bottom.

The network forms a closed loop.

$$\text{Network Diameter, } D = 2(k/2)$$

Wrap-around links reduce the longest distance. Thus Torus generally has lower average latency than Mesh because packets can take shorter wrap-around paths.

Routing

Booksim primarily implements **Dimension-Order Routing (DOR)** for the KNCube topology. A packet is routed by resolving one coordinate (dimension) at a time until it reaches the destination.

For a 2D KNCube ($n = 2$), routing proceeds in the order:

X -> Y

This is commonly called XY routing.

In a Torus, each dimension is cyclic because opposite edges are connected.

For every dimension, the router computes **two possible distances**:

Forward distance

Wrap-around (reverse) distance

It chooses the **shorter** one.

❖ Mesh

Because channels are always acquired in the fixed order

X->Y

no cyclic channel dependency can form.

Therefore,

Dimension-Order Routing is deadlock-free on a Mesh.

❖ Torus

A Torus contains **cyclic links** due to wrap-around connections.

Even with Dimension-Order Routing, packets can form circular channel dependencies.

To eliminate these cycles, Booksim uses **Virtual Channels (VCs)**.

Packets change to another VC when crossing certain wrap-around links, breaking the dependency cycle.

Therefore,

Dimension-Order Routing on a Torus requires Virtual Channels to guarantee deadlock freedom.

Advantages

Mesh	Torus
Simple architecture	Lower average latency
Easy to implement	Smaller network diameter
Regular layout	Better throughput under heavy traffic
Short physical links	Better load balancing due to multiple shortest paths
Lower wiring complexity	Reduced average hop count

Limitations

Mesh	Torus
Larger average hop count	Long wrap-around links increase wiring complexity
Higher latency for distant nodes	Larger router complexity
Lower throughput under heavy load	More difficult floorplanning in hardware
Congestion near network center	Higher implementation cost
Performance degrades as network grows	Deadlock avoidance requires virtual channels

Conclusion:

The **KNCube topology** provides a regular and scalable interconnection network from which both Mesh and Torus are derived. Mesh offers a simple and hardware-friendly design with straightforward routing, making it suitable for many on-chip networks. Torus enhances the same structure by introducing wrap-around links, reducing the average hop count and network diameter, which generally improves latency and throughput. These performance gains come at the cost of increased wiring complexity and more sophisticated deadlock management. In Booksim, both topologies share the same implementation (**KNCube**) and differ only in whether edge routers are connected (Torus) or left open (Mesh), allowing direct comparison of the trade-offs between simplicity and communication efficiency.

Feature	KNCube (Mesh/Torus)	Dragonfly
Connection style	Regular grid	Groups of routers
Scalability	Moderate	Very high
Average hops	Higher	Lower
Wiring	Simple	Complex
Routing	Mostly DOR	Minimal / adaptive routing options
Best for	Many-core chips, on-chip NoCs	Large HPC systems and scalable fabrics
Physical layout	Easy	More difficult
Main strength	Simplicity and predictability	Reduced global communication cost